The TAGE Branch Predictor: Design and Code

A short guide to how the predictor works and what each SystemVerilog file does

> **Project:** TAGE branch predictor in SystemVerilog, running on the Digilent Nexys A7-100T FPGA board.
>
> **Contents**
>
> 1. How TAGE works
> 2. The design: block diagram and schematics (Fig. A, B, C)
> 3. Reading the code: a SystemVerilog cheat-sheet
> 4. The predictor code, line by line (inputs, outputs and every statement)
> 5. The board demo, line by line
> 6. Testing and results

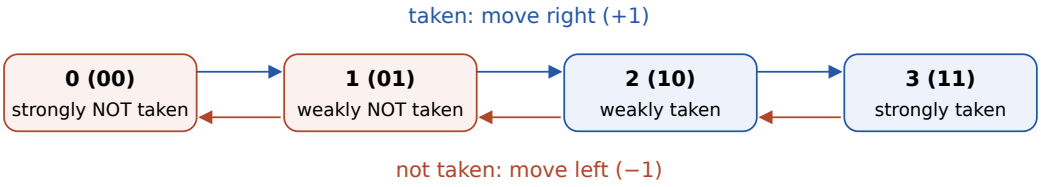
Member	Files
Member 1	tage_pkg.sv, tage_table.sv
Member 2	tage_hash.sv, tage_predict.sv
Member 3	tage_update.sv, tage_top.sv
Member 4	tage_ref.sv (reference model), testing
Board	nexys7_top.sv, trace_rom.sv, branch_trace.mem, perf_counters.sv, seg7_driver.sv

1. How TAGE works

1.1 The problem

A **branch** is an `if` or a loop check in a program: the CPU either jumps (**taken**) or continues (**not taken**). The CPU only knows the answer a few cycles later, so it **guesses** to keep working. A wrong guess wastes cycles. A **branch predictor** makes that guess, using the branch's address (the **PC**) and what happened in the past.

1.2 Building block 1: the saturating counter



A 2-bit saturating counter. The left bit (the MSB) is the prediction: 0 = not taken, 1 = taken.

Taken: +1, not taken: −1, but it never goes past the ends. The top bit (MSB) is the prediction. One surprise doesn't flip it, so a loop that is taken 99 times and then not taken once is still predicted well. TAGE's tagged tables use the same idea with 3-bit counters (0–3 = not taken, 4–7 = taken).

1.3 Building block 2: global history (GHR)

Many branches depend on what earlier branches did, e.g. `if (x) ... if (x) ...`. The **global history register** is a 32-bit shift register holding the last 32 outcomes (1 = taken). After every branch the newest outcome is shifted in at bit 0 and the oldest drops out.

The problem: some branches need only a little history, others need a lot. Too little and you miss the pattern; too much and learning is slow. **TAGE uses several tables, each with a different amount of history, and picks the best one for each branch.**

1.4 The five tables

Table	Entries	History used	Each entry holds
T0 (base)	1024	none (PC only)	2-bit counter
T1	256	last 4 outcomes	**valid**: entry in use? **tag** (8 bits): fingerprint of which branch + history owns this entry **ctr** (3 bits): prediction counter **u**: "useful" bit, protects good entries
T2	256	last 8	
T3	256	last 16	
T4	256	last 32	

History lengths 4, 8, 16, 32 double each time (a geometric series). Total storage: 1024×2 + 4×256×13 = **15,360 bits**.

Why tags? Only 256 entries but many more (branch, history) combinations, so several land on the same entry. The tag is checked on every read: the entry is used only if it is valid *and* its tag

matches (a **hit**). Otherwise that table is ignored.

1.5 The hash: PC + history → index and tag

```
Hash for tagged table i (history length L)
```

```
f       = xor_fold(ghr[L-1:0], 8)     // squeeze the last L history bits into 8 bits
idx[i]  = pc[9:2]   XOR  f            // which of the 256 entries to read
tag[i]  = pc[17:10] XOR  rotl(f, 1)   // fingerprint to compare with the stored tag
```

XOR fold: cut the history into 8-bit pieces and XOR them together. **rotl** = rotate left by one bit, so the index and the tag are different mixes of the same bits. Example: PC = 0x1A3C and GHR = 0xB6CE5D36 give T3 (L = 16): f = 0x5D ^ 0x36 = 0x6B, idx = 0x8F ^ 0x6B = **0xE4**, tag = 0x06 ^ 0xD6 = **0xD0** (Fig. B shows this as gates).

1.6 Predicting

1. Read all five tables at once, each at its own index.
2. **Provider** = the hitting table with the **longest** history (T4 > T3 > T2 > T1). No hit → T0.
3. **Prediction** = MSB of the provider's counter.
4. **Alternate** = the second-longest hit (or T0): what we would have predicted without the provider.
5. Save everything used (provider, alternate, indices, tags, both predictions) in the **meta** bundle, a "receipt" so the update knows which entries to change.

1.7 Learning: the four update rules

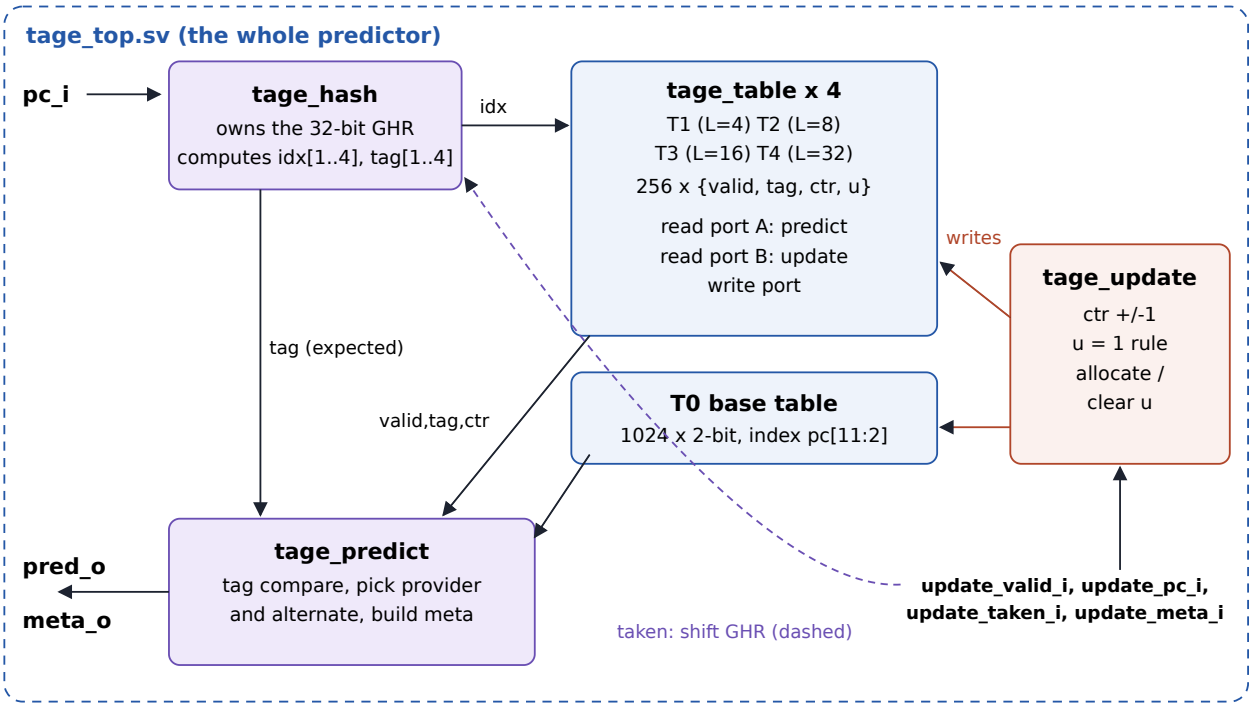
1. **Counter:** provider's counter +1 if taken, −1 if not (stops at 7 / 0). If the provider was T0, train T0.
2. **Useful bit:** if the provider was right and the alternate was wrong, set the provider's `u = 1`.
3. **Allocate on a mistake:** if the prediction was wrong and the provider is not T4, take the *first* longer table whose entry has `u = 0` and write a new entry there (valid = 1, new tag, counter 4 if taken / 3 if not, u = 0). If all longer entries have `u = 1`, write nothing but clear their u bits so there is room next time.
4. **History:** shift the real outcome into the GHR.

> **The idea in one line:** every branch starts in T0; each time it is mispredicted it gets an entry in a table with more history, so it climbs until it reaches the smallest history that predicts it right.

2. The design

One branch takes **two clock cycles**:

Cycle	What happens
A: predict	`pc_i` goes in; hash → table reads → tag compare → provider choice, all in one cycle. `pred_o` and `meta_o` come out and are saved in a register.
B: update	The real outcome and the saved meta go back in. At the clock edge the tables and the GHR are written.



Inside tage_top: who talks to whom. Purple = Member 2, blue = Member 1, red = Member 3.

File	Job
`tage_pkg.sv`	shared constants, the meta type, `xor_fold` / `rotl`
`tage_table.sv`	one table: 2 read ports + 1 write port
`tage_hash.sv`	GHR + index/tag hash for T1..T4
`tage_predict.sv`	hits, provider, alternate, prediction, meta
`tage_update.sv`	the four rules → what to write into each table
`tage_top.sv`	the whole predictor: connects all of the above + T0

Fig. A: the complete TAGE datapath

How to read it: signals flow left to right. **Black** = the prediction (all in one clock cycle): pc_i goes into tage_hash, which produces an index and a tag for each table. Each table is read at its index, its stored tag is compared (**=**) with the computed tag, and the AND gate gives *hit*. The priority encoders pick the provider and alternate; the muxes (**M**) select their counter MSBs. Everything is packed into *meta*. The **pipeline register** keeps pc, meta and the real outcome for one cycle. Then **orange** = tage_update writes the tables, **purple** = the entries being updated are read back first, **green** = the real outcome is shifted into the GHR.

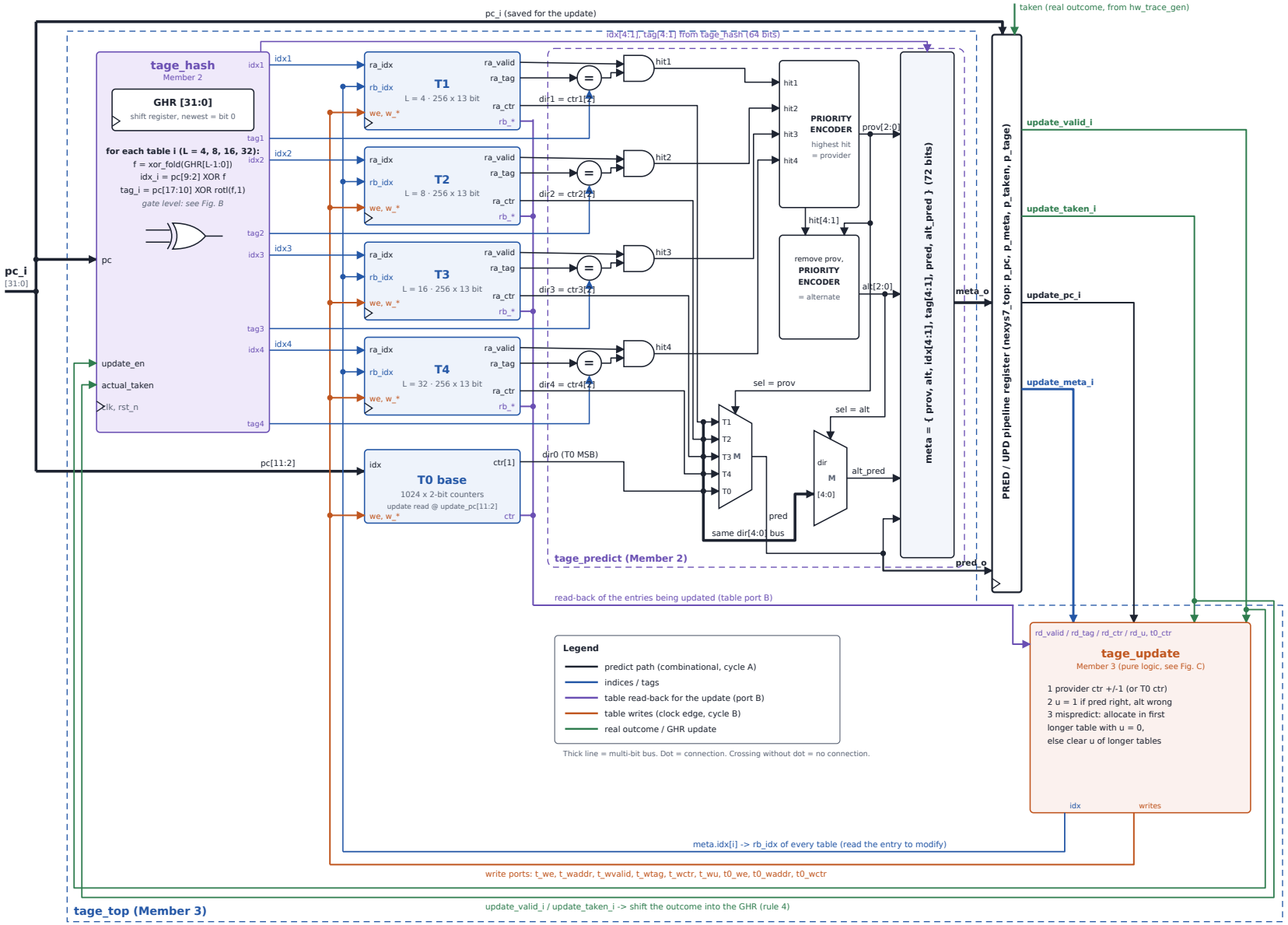
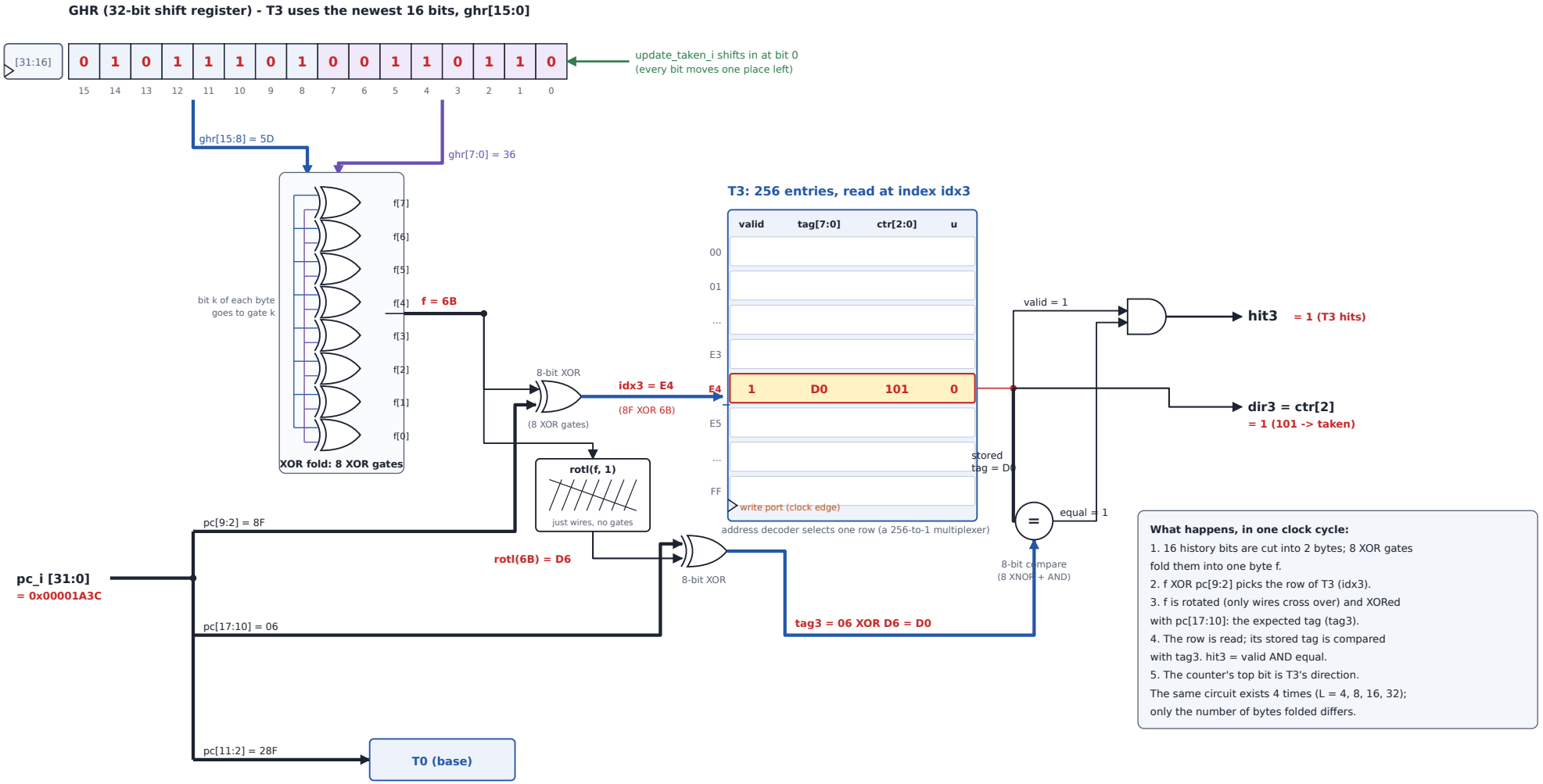


Fig. B: inside one table lookup, at gate level (T3, history length 16)

How to read it: top: the 16 newest history bits (red = the worked example). The two bytes are XORed bit by bit by 8 XOR gates (the "fold"), giving **f**. f XOR pc[9:2] (another 8 XOR gates) selects the table row. f rotated by one place (only wires crossing, no gates) XOR pc[17:10] gives the expected tag. The selected row's stored tag is compared with it; hit = valid AND equal; the direction is the counter's top bit. T1, T2 and T4 have the same circuit, folding 4 bits, 1 byte or 4 bytes instead.

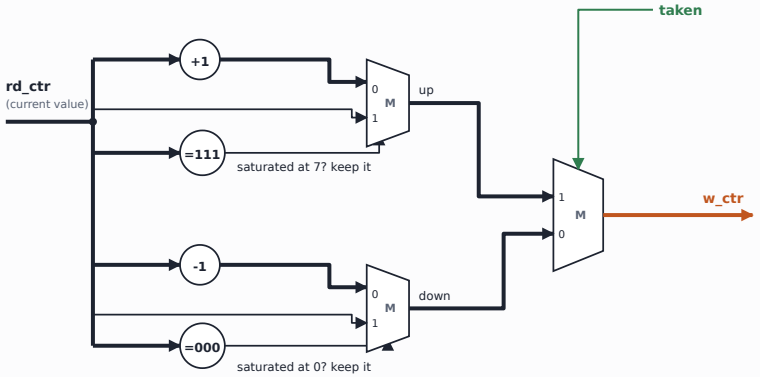


Red numbers: the worked example from chapter 3 (GHR = B6CE5D36, pc = 0x1A3C).

Fig. C: the update logic (tage_update) at gate level

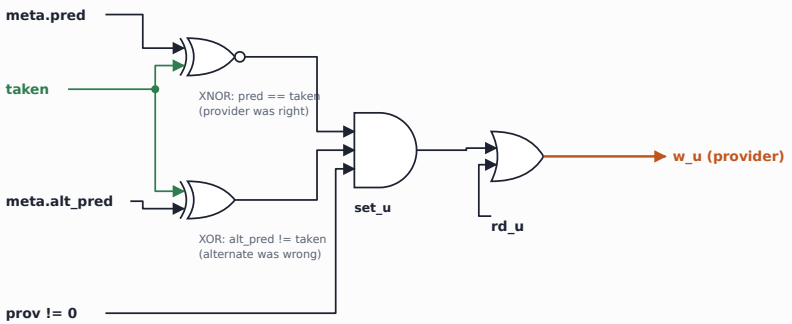
How to read it: this is all combinational logic that decides *what to write*; the tables store it at the next clock edge. **Rule 1**: the counter is incremented or decremented, but a comparator keeps it from going past 7 or 0; the *taken* mux chooses up or down. **Rule 2**: XNOR/XOR gates check "provider right, alternate wrong" to set u. **Rule 3**: on a misprediction, table j is a candidate if it is longer than the provider and its u = 0; the priority gates (inverted input = bubble) make sure only the *first* candidate is allocated; if there is no candidate, u is cleared instead.

Rule 1: saturating counter of the provider (3-bit)



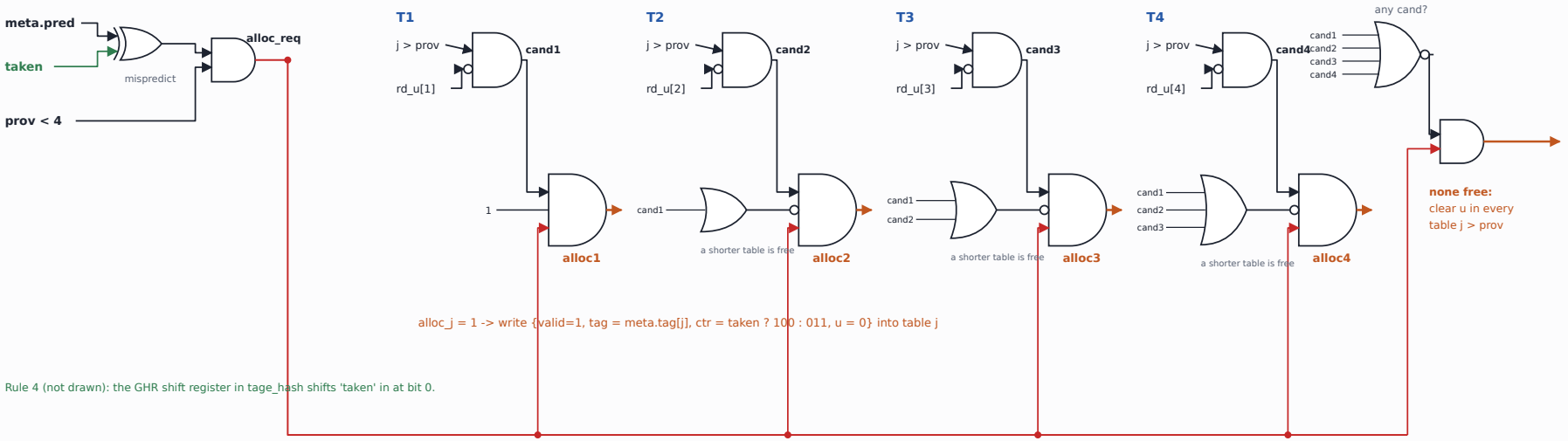
Example: ctr = 5 (101), taken -> 6 (110). ctr = 7, taken -> stays 7.
Written to table 'prov' (we = 1). If prov = 0 the same circuit (2-bit) updates T0.

Rule 2: useful bit (u) of the provider



u becomes 1 only when the provider made the difference: it was right and the alternate would have been wrong. Otherwise u keeps its old value.

Rule 3: allocation after a misprediction (first longer table with u = 0)



Rule 4 (not drawn): the GHR shift register in tage_hash shifts 'taken' in at bit 0.

The inverted input (bubble) on each priority gate blocks table j if an earlier (shorter) table already got the entry, so at most one table is allocated.

3. Reading the code: a cheat-sheet

Everything you need to read the files. Remember: this code describes **hardware**. All lines exist at the same time as wires and flip-flops; they are not run one after another like C.

You see	It means
`module name ( ... ); ... endmodule`	A circuit block. The list in brackets are its pins (ports).
`input logic [7:0] x`	An input pin `x` that is 8 wires wide (bits 7 down to 0). No `[ ]` = 1 wire.
`output logic y`	An output pin.
`logic [7:0] s;` / `wire` / `reg`	An internal signal (a bundle of wires). `wire` and `reg` are older names for the same thing.
`logic [255:0][2:0] mem;`	An array: 256 items of 3 bits each. `mem[5]` is item number 5.
`parameter int N = 8`	A constant, fixed when the circuit is built. Can be changed per copy of the module.
`assign a = b & c;`	A permanent connection: `a` always equals `b AND c`. Pure logic, no clock.
`always_comb begin ... end` `always @* begin ... end`	Logic written with if/case/for. Still pure logic (no memory): outputs change as soon as inputs change.
`always_ff @(posedge clk ...)`	**Flip-flops** (memory). The values inside change only at the rising edge of the clock.
`... or negedge rst_n` + `if (!rst_n)`	Reset: when the reset wire goes to 0, the memory is cleared immediately. `_n` means "active when 0".
`a <= b;` vs `a = b;`	`<=` is used for flip-flops (stored at the clock edge). `=` is used in combinational logic.
`8'b0000_0011`, `3'd4`, `8'hA5`, `'0`, `'1`	Numbers: bits'base value (b = binary, d = decimal, h = hex). `'0` = all bits 0, `'1` = all bits 1.
`{a, b}`	Glue signals side by side into a wider one. `{4{1'b1}}` = 1111 (repeat).
`x[9:2]` `x[2 +: 8]`	Pick bits 9 down to 2. `+:` = "8 bits starting at bit 2" (the same thing).
`^` `&` `|` `~` `!` `&&`	XOR, AND, OR, NOT (bitwise), NOT (true/false), logical AND.
`c ? x : y`	A multiplexer: if `c` then `x` else `y`.
`name #(.P(4)) u_name ( .pin(signal), ... );`	Place a copy (instance) of module `name` called `u_name`, set its parameter `P` to 4, and connect its pin `pin` to our `signal`.
`generate for (...)`	Build several copies of hardware with a loop (decided when the circuit is built).
`for` inside always blocks	Not a loop in time: the tool copies the body once per value ("unrolls" it) into parallel hardware.
`package ... import pkg::*;`	A shared file of constants/types, and the line that lets a module use them.

`typedef struct packed {...} t;`	A named bundle of fields stored as one long bit-vector (like a C struct).

4. The predictor code, file by file

4.1 tage_pkg.sv: shared definitions Member 1

Job: holds the numbers and types that every other file uses. It has no inputs or outputs and builds no hardware by itself.

`rtl/tage_pkg.sv`

`package tage_pkg;`	Start of the shared package. Other files use it with `import tage_pkg::*;`
`parameter int NUM_T = 4;`	There are 4 tagged tables (T1..T4).
`parameter int HIST_LEN[4] = '{4, 8, 16, 32};`	How many history bits each tagged table uses: T1 = 4, T2 = 8, T3 = 16, T4 = 32.
`parameter int IDX_W = 8;`	An index into a tagged table is 8 bits, so each tagged table has $2^8 = 256$ entries.
`parameter int TAG_W = 8;`	Each stored tag is 8 bits.
`parameter int CTR_W = 3;`	Each tagged counter is 3 bits (values 0..7).
`parameter int BASE_IDX_W = 10;`	T0's index is 10 bits, so T0 has $2^{10} = 1024$ entries.

`typedef struct packed {`
`  logic [2:0] prov;`
`  logic [2:0] alt;`
`  logic [4:1][7:0] idx;`
`  logic [4:1][7:0] tag;`
`  logic pred;`
`  logic alt_pred;`
`} tage_meta_t;`
	Defines the **meta** bundle (the "receipt" of a prediction), 72 bits in total:
	`prov` : which table gave the prediction (0 = T0, 1..4 = T1..T4)
	`alt` : the alternate table
	`idx[1..4]` : the 4 table indices that were used
	`tag[1..4]` : the 4 tags that were computed
	`pred` : the prediction (1 = taken)
	`alt_pred` : what the alternate table predicted

`function automatic [7:0] xor_fold(`
`  input logic [31:0] data, input int length);`
`  logic [7:0] folded;`
`  folded = '0;`
	A helper that squeezes the first `length` bits of `data` into 8 bits. Starts with 8 zeros.

`  for (int i = 0; i < length; i++) begin`
`  folded[i % 8] ^= data[i];`
`  end`
`  return folded;`
	For every history bit i: XOR it into output bit `i % 8` (the remainder of i / 8). So bits 0, 8, 16, 24 all land on output bit 0, bits 1, 9, 17, 25 on bit 1, and so on. In hardware this is just XOR gates.

`function automatic [7:0] rotl(`
`  input logic [7:0] val, input int shift);`
`  logic [15:0] dbl;`
`  dbl = {val, val} << (shift % 8);`
`  return dbl[15:8];`
	Rotate an 8-bit value left. Two copies are glued together (16 bits) and shifted left; the top 8 bits are then the rotated value (bits that fall off the left come back on the right). `rotl(1000_0001, 1) = 0000_0011`.

4.2 tage_table.sv: one predictor table Member 1

Job: a memory of 256 entries, each `{valid, tag, ctr, u}`. It can show two entries at the same time (read ports A and B) and change one entry per clock (write port). T1..T4 are 4 copies of this module.

Name	In / Out	Bits	What it carries
`clk`	in	1	the clock
`rst_n`	in	1	reset: 0 = clear the whole table
`ra_idx`	in	8	**Port A** (predict): which entry to look at
`ra_valid, ra_tag, ra_ctr, ra_u`	out	1, 8, 3, 1	the contents of entry `ra_idx`
`rb_idx`	in	8	**Port B** (update): which entry to look at
`rb_valid, rb_tag, rb_ctr, rb_u`	out	1, 8, 3, 1	the contents of entry `rb_idx`
`we`	in	1	write enable: 1 = write at the next clock edge
`w_idx`	in	8	which entry to write
`w_valid, w_tag, w_ctr, w_u`	in	1, 8, 3, 1	the new contents for that entry

rtl/tage_table.sv: parameters	

`module tage_table #(`
`  parameter int IDX_W = 8,`
`  parameter int TAG_W = 8,`
`  parameter int CTR_W = 3,`
	Sizes of the table, with default values. `tage_top` uses the defaults.

`parameter logic [CTR_W-1:0] CTR_INIT = {1'b0, {(CTR_W-1){1'b1}}},`	Counter value after reset: a 0 followed by ones = `011` = 3 = "weakly not taken".
`localparam int TW = (TAG_W > 0) ? TAG_W : 1`	Width of the tag pins. A pin cannot be 0 bits wide, so if the table has no tag (TAG_W = 0) it is 1 bit and unused.
`localparam int DEPTH = 1 << IDX_W;`	Number of entries: 1 shifted left 8 times = 256.

rtl/tage_table.sv: the counters	
`logic [DEPTH-1:0][CTR_W-1:0] ctr_mem;`	The counter memory: 256 counters of 3 bits.
`always_ff @(posedge clk or negedge rst_n) begin`	Flip-flops: change at the clock edge, or immediately when reset goes to 0.

`  if (!rst_n) begin`
`  for (int k = 0; k < DEPTH; k++)`
`  ctr_mem[k] <= CTR_INIT;`
	During reset: set **all 256** counters to 011.

`  end else if (we) begin`
`  ctr_mem[w_idx] <= w_ctr;`
`  end`
	Otherwise, if write enable is 1: entry number `w_idx` gets the new counter `w_ctr`. All other entries keep their value.

`assign ra_ctr = ctr_mem[ra_idx];` `assign rb_ctr = ctr_mem[rb_idx];`	Reading: port A always shows the counter at `ra_idx`, port B the counter at `rb_idx`. This is a big multiplexer, no clock: the output changes as soon as the index changes.

rtl/tage_table.sv: valid, tag and u bits	
`generate` `  if (TAG_W > 0) begin : g_tagged`	Only if the table has tags (true for T1..T4), build the following hardware. Decided when the circuit is built.

`logic [DEPTH-1:0]  valid_mem;`
`logic [DEPTH-1:0]  u_mem;`
`logic [DEPTH-1:0][TW-1:0] tag_mem;`
	256 valid bits, 256 useful bits, 256 tags of 8 bits.

`always_ff @(posedge clk or negedge rst_n) begin`
`  if (!rst_n) begin`
`  for (int k = 0; k < DEPTH; k++) begin`
`  valid_mem[k] <= 1'b0;`
`  u_mem[k]  <= 1'b0;`
`  tag_mem[k]  <= '0;`
`  end`
	Reset: every entry becomes empty (valid = 0, u = 0, tag = 0).

`  end else if (we) begin`
`  valid_mem[w_idx] <= w_valid;`
`  u_mem[w_idx]  <= w_u;`
`  tag_mem[w_idx]  <= w_tag;`
`  end`
	Write: same as the counter: the entry at `w_idx` gets the new valid, u and tag. So one write changes the whole entry {valid, tag, ctr, u} at once.

`assign ra_valid = valid_mem[ra_idx];`
`assign ra_tag  = tag_mem[ra_idx];`
`assign ra_u  = u_mem[ra_idx];`
`assign rb_valid = valid_mem[rb_idx];  ...`
	Read ports A and B for valid, tag and u, the same way as the counter.

`end else begin : g_base`
`  assign ra_valid = 1'b1;`
`  assign ra_tag  = '0;`
`  assign ra_u  = 1'b0;  ...`
	If the table has no tags (TAG_W = 0, a base table): no memory is built for them. Valid always reads 1, tag 0, u 0.

4.3 tage_hash.sv: history register and hash Member 2

Job: stores the global history (GHR) and computes, for T1..T4, which entry to read (`idx`) and which tag to expect (`tag`).

Name	In / Out	Bits	What it carries
`clk, rst_n`	in	1, 1	clock and reset
`pc`	in	32	address of the branch being predicted
`idx`	out	32	4 indices glued together: bits 7..0 = T1, 15..8 = T2, 23..16 = T3, 31..24 = T4
`tag`	out	32	4 expected tags, same order
`update_en`	in	1	1 = a branch finished: shift its outcome into the history
`actual_taken`	in	1	the real outcome of that branch (1 = taken)

rtl/tage_hash.sv	

`localparam GHR_W  = 32;`
`localparam FOLD_W  = 8;`
`localparam PC_IDX_LSB = 2;`
`localparam PC_TAG_LSB = PC_IDX_LSB + IDX_W;`
	Constants: history is 32 bits, folds are 8 bits, the index uses PC bits starting at bit 2 (`pc[9:2]`), the tag uses PC bits starting at bit 10 (`pc[17:10]`). Bits 1..0 are skipped because instructions are 4 bytes apart, so they are always 0.

`reg [GHR_W-1:0] ghr;`	The 32-bit global history register.

`always @(posedge clk or negedge rst_n) begin`
`  if (!rst_n)`
`  ghr <= {GHR_W{1'b0}};`
	Reset: history = 32 zeros.

`  else if (update_en)` `  ghr <= {ghr[GHR_W-2:0], actual_taken};`	When a branch finishes: keep bits 30..0, move them one place left, put the new outcome at bit 0. The oldest bit (31) drops out. So `ghr[0]` is always the newest outcome.

`function [FOLD_W-1:0] xor_fold(input [GHR_W-1:0] h,`
`  input integer len);`
`  ...`
`  for (b = 0; b < GHR_W; b = b + 1)`
`  if (b < len)`
`  xor_fold[b % FOLD_W] = xor_fold[b % FOLD_W] ^ h[b];`
	Member 2's own copy of the fold (same result as the package one): XOR history bit b into output bit b % 8, for the first `len` bits.

`wire [FOLD_W-1:0] fold1 = xor_fold(ghr, HIST_LEN_1);`
`wire [FOLD_W-1:0] fold2 = xor_fold(ghr, HIST_LEN_2);`
`wire [FOLD_W-1:0] fold3 = xor_fold(ghr, HIST_LEN_3);`
`wire [FOLD_W-1:0] fold4 = xor_fold(ghr, HIST_LEN_4);`
	Four 8-bit summaries of the history: of the last 4, 8, 16 and 32 outcomes. Each one becomes a fixed tree of XOR gates (Fig. B).

`wire [IDX_W-1:0] pc_idx = pc[PC_IDX_LSB +: IDX_W];` `wire [TAG_W-1:0] pc_tag = pc[PC_TAG_LSB +: TAG_W];`	Take 8 PC bits for the index (`pc[9:2]`) and the next 8 for the tag (`pc[17:10]`).

`assign idx = { pc_idx ^ fold4,`
`  pc_idx ^ fold3,`
`  pc_idx ^ fold2,`
`  pc_idx ^ fold1 };`
	The 4 indices: PC bits XOR the fold of each table. Glued together, T4 on the left (top bits), T1 on the right.

`assign tag = { pc_tag ^ {fold4[FOLD_W-2:0], fold4[FOLD_W-1]},`
`  ... ,`
`  pc_tag ^ {fold1[FOLD_W-2:0], fold1[FOLD_W-1]} };`
	The 4 tags: PC tag bits XOR the fold rotated left by 1. `{f[6:0], f[7]}` moves bit 7 to the right end, which is the rotation (just wires crossing, no gates).

`// synthesis translate_off`
`initial begin`
`  if (IDX_W != FOLD_W ...) $display("ERROR ...");`
`...`
`// synthesis translate_on`
	Simulation only, not hardware: prints an error if someone sets wrong sizes.

4.4 tage_predict.sv: making the prediction Member 2

Job: pure logic (no clock, no memory). Looks at what the tables returned, finds which tables hit, picks the provider and alternate, and outputs the prediction and the meta bundle.

Name	In / Out	Bits	What it carries
`t0_ctr_msb`	in	1	top bit of T0's counter = T0's prediction
`t_valid`	in	4	valid bit read from T1..T4 (port A)
`t_tag`	in	32	stored tags read from T1..T4 (8 bits each)
`t_ctr`	in	12	counters read from T1..T4 (3 bits each)
`t_u`	in	4	useful bits (not used here; kept so the ports match the plan)
`hash_idx, hash_tag`	in	32, 32	the indices and expected tags from `tage_hash`
`pred`	out	1	the prediction: 1 = taken
`meta`	out	72	the receipt: {prov, alt, idx, tag, pred, alt_pred}

`rtl/tage_predict.sv`

`function [2:0] highest_hit(input [4:1] v);`
`  casez (v)`
`  4'b1???: highest_hit = 3'd4;`
`  4'b01??: highest_hit = 3'd3;`
`  4'b001?: highest_hit = 3'd2;`
`  4'b0001: highest_hit = 3'd1;`
`  default: highest_hit = 3'd0;`
`  endcase`
	Given 4 hit bits (T4..T1), return the number of the highest table that hit. `?` means "don't care". If T4 hit, answer 4 whatever the others are; else if T3 hit, 3; and so on; if none hit, 0 (= T0). In hardware this is a **priority encoder**.

`reg [4:1] hit;`
`reg [4:1] hit_below;`
`reg [4:0] dir;`
	`hit[t]`: did table t hit? `dir[t]`: what does table t predict (dir[0] is T0)? `hit_below`: the hits without the provider.

`always @* begin` `  dir[0] = t0_ctr_msb;`	Combinational logic. T0's prediction is its counter's top bit.

`  for (t = 1; t <= 4; t = t + 1) begin`
`    hit[t] = t_valid[t] &&`
`      (t_tag[(t-1)*TAG_W +: TAG_W] == hash_tag[(t-1)*TAG_W +: TAG_W]);`
	For each table: **hit = valid AND (stored tag == expected tag)**. `(t-1)*TAG_W +: TAG_W` picks table t's 8 bits out of the 32-bit bus (T1 = bits 7..0, T2 = 15..8, ...). Hardware: an 8-bit comparator and an AND gate per table.

`    dir[t] = t_ctr[(t-1)*CTR_W + CTR_W - 1];`	Table t's prediction = the top bit of its 3-bit counter (1 if the counter is 4..7).
`assign prov = highest_hit(hit);`	**Provider** = the highest table that hit.

`always @* begin`
`  hit_below = hit;`
`  if (prov != 3'd0)`
`    hit_below[prov] = 1'b0;`
`end`
`assign alt = highest_hit(hit_below);`
	Alternate: copy the hits, erase the provider's hit, and take the highest again. That is the second-longest hit (or 0 = T0 if there is none).

`assign pred     = dir[prov];` `assign alt_pred = dir[alt];`	The prediction is the provider's direction; alt_pred is the alternate's. If prov = 0, `dir[0]` = T0 is used automatically. Hardware: two multiplexers.
`assign meta = {prov, alt, hash_idx, hash_tag, pred, alt_pred};`	Glue everything into the 72-bit receipt, in the same order as `tage_meta_t`.

4.5 tage_update.sv: the learning rules Member 3

Job: pure logic. When a branch finishes, it decides **what to write** into T0..T4. It stores nothing; the tables do the writing at the next clock edge.

Name	In / Out	Bits	What it carries
`update_valid`	in	1	1 = a branch finished, apply the rules
`update_pc`	in	32	the address of that branch (to find its T0 entry)
`taken`	in	1	its real outcome
`meta`	in	72	the receipt saved when it was predicted
`rd_valid, rd_tag, rd_ctr, rd_u`	in	4, 4×8, 4×3, 4	current contents of T1..T4 at the indices in the receipt (from read port B)
`t0_ctr`	in	2	current T0 counter of that branch
`t0_we, t0_waddr, t0_wctr`	out	1, 10, 2	write to T0: enable, which entry, new counter
`t_we`	out	4	write enable for T1..T4 (one bit each)
`t_waddr`	out	4×8	which entry to write in each table
`t_wvalid, t_wtag, t_wctr, t_wu`	out	4, 4×8, 4×3, 4	the new entry contents for each table

```
rtl/tage_update.sv: defaults
```

`always_comb begin` `  prov = int'(meta.prov);`	Combinational logic. `prov` = the provider table number from the receipt.

`  t0_we    = 1'b0;`
`  t0_waddr = update_pc[BASE_IDX_W+1:2];`
`  t0_wctr  = t0_ctr;`
	Default for T0: do not write. If it is written, the address is the branch's T0 entry (`pc[11:2]`) and the value stays the same unless a rule changes it.

`  for (int i = 1; i <= 4; i++) begin`
`    t_we[i]     = 1'b0;`
`    t_waddr[i]  = meta.idx[i];`
`    t_wvalid[i] = rd_valid[i];`
`    t_wtag[i]   = rd_tag[i];`
`    t_wctr[i]   = rd_ctr[i];`
`    t_wu[i]     = rd_u[i];`
`  end`
`  found = 1'b0;`
	Defaults for T1..T4: no write; the address is the same entry the prediction used; the new values = the current values. So when a rule only changes, say, the counter, the other fields are rewritten unchanged (**read-modify-write**). `found` is cleared (used by rule 3).

`  if (update_valid) begin`	Everything below only happens when a branch actually finished.

`rtl/tage_update.sv: rules 1 and 2`

`if (prov != 0) begin` `  t_we[prov] = 1'b1;`	**Rule 1.** If a tagged table was the provider: write to it.

`  if (taken) begin`
`    if (rd_ctr[prov] != {CTR_W{1'b1}})`
`      t_wctr[prov] = rd_ctr[prov] + 1'b1;`
	Taken: counter + 1, unless it is already 111 (= 7, the maximum).

`  end else begin`
`    if (rd_ctr[prov] != {CTR_W{1'b0}})`
`      t_wctr[prov] = rd_ctr[prov] - 1'b1;`
`  end`
	Not taken: counter − 1, unless it is already 000 (the minimum). This is the **saturating** counter.

`  if (meta.pred == taken && meta.alt_pred != taken)` `    t_wu[prov] = 1'b1;`	**Rule 2.** The provider was right and the alternate would have been wrong: mark the entry useful (u = 1).

`end else begin`
`  t0_we = 1'b1;`
`  if (taken) begin`
`    if (t0_ctr != 2'b11) t0_wctr = t0_ctr + 2'd1;`
`  end else begin`
`    if (t0_ctr != 2'b00) t0_wctr = t0_ctr - 2'd1;`
`  end`
`end`
	No tagged provider (prov = 0): train T0's 2-bit counter the same way (limits 3 and 0).

`rtl/tage_update.sv: rule 3 (allocation)`

`if (meta.pred != taken && prov < 4) begin`	**Rule 3.** Only if the prediction was **wrong** and there is a longer table than the provider (provider is not T4).
`  for (int i = 1; i <= 4; i++) begin` `    if (i > prov && !found && !rd_u[i]) begin`	Go through T1..T4. Pick table i if it is longer than the provider, nothing was picked yet (`!found`) and its entry is not useful (u = 0). So the **first** (shortest) free longer table is picked.

`      found       = 1'b1;`
`      t_we[i]     = 1'b1;`
`      t_wvalid[i] = 1'b1;`
`      t_wtag[i]   = meta.tag[i];`
`      t_wctr[i]   = taken ? 3'b100 : 3'b011;`
`      t_wu[i]     = 1'b0;`
	Take over that entry: valid = 1, tag = the tag this branch had, counter = weakly towards the real outcome (100 = 4 if taken, 011 = 3 if not), u = 0. `found = 1` stops other tables from being picked.

`  if (!found) begin`
`    for (int i = 1; i <= 4; i++)`
`      if (i > prov) begin`
`        t_we[i] = 1'b1;`
`        t_wu[i] = 1'b0;`
`      end`
	If every longer table's entry was useful (nothing free): do not take any entry, but clear u in all of them, so that next time one is free.

`(rule 4 is in tage_hash)`	**Rule 4** (shift the outcome into the GHR) happens in `tage_hash`, which receives the same update signals.

4.6 tage_top.sv: the complete predictor Member 3

Job: places the hash, the base table T0, four tagged tables, the predict logic and the update logic, and connects them (see the block diagram in chapter 2).

Name	In / Out	Bits	What it carries
`clk, rst_n`	in	1, 1	clock and reset
`pc_i`	in	32	**predict:** the address of the branch to predict
`pred_o`	out	1	**predict:** the guess (1 = taken), in the same cycle
`meta_o`	out	72	**predict:** the receipt; the user must keep it until the branch finishes
`update_valid_i`	in	1	**update:** 1 = a branch finished this cycle
`update_pc_i`	in	32	**update:** its address
`update_taken_i`	in	1	**update:** its real outcome
`update_meta_i`	in	72	**update:** the receipt that was given out when it was predicted

`rtl/tage_top.sv`

```
logic [4*IDX_W-1:0] hash_idx;
logic [4*TAG_W-1:0] hash_tag;
tage_hash #( ... ) u_hash (
  .clk(clk), .rst_n(rst_n), .pc(pc_i),
  .idx(hash_idx), .tag(hash_tag),
  .update_en(update_valid_i),
  .actual_taken(update_taken_i) );
```

Place the hash. It gets the PC to predict and gives back 4 indices and 4 tags. It also gets the update signals, so the real outcome is shifted into the GHR (rule 4).

```
logic [(1<<BASE_IDX_W)-1:0][1:0] T0;
```

The base table T0: 1024 counters of 2 bits, built right here (not with tage_table).

```
assign t0_ctr_a = T0[pc_i[BASE_IDX_W+1:2]];
assign t0_ctr_b = T0[update_pc_i[BASE_IDX_W+1:2]];
```

Two reads of T0: `_a` for the branch being predicted (index `pc_i[11:2]`), `_b` for the branch being updated.

```
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)
    for (int k = 0; k < (1<<BASE_IDX_W); k++) T0[k]
<= 2'b10;
  else if (t0_we)
    T0[t0_waddr] <= t0_wctr;
end
```

T0's flip-flops. Reset: all 1024 counters = 10 (weakly taken). Otherwise, when tage_update says so, write one counter.

```
logic [4:1] a_valid, b_valid, a_u, b_u;
logic [4:1][TAG_W-1:0] a_tag, b_tag;
logic [4:1][CTR_W-1:0] a_ctr, b_ctr;
logic [4:1] t_we, t_wvalid, t_wu; ...
```

Wires between the parts: `a_*` = what the 4 tables show on port A, `b_*` = port B, `t_w*` = the write commands from tage_update.

```
generate
  for (g = 1; g <= 4; g++) begin : g_tab
    tage_table #( ... ) u_tab (
```

Build 4 copies of tage_table: T1, T2, T3, T4.

```
      .ra_idx  (hash_idx[(g-1)*IDX_W +: IDX_W]),
      .ra_valid(a_valid[g]), .ra_tag(a_tag[g]),
      .ra_ctr(a_ctr[g]), .ra_u(a_u[g]),
```

Port A of table g reads at the index from the hash (the **new** branch being predicted). Its contents go to `a_*[g]`.

`.rb_idx  (update_meta_i.idx[g]),` `.rb_valid(b_valid[g]), ... ,`	Port B reads at the index saved in the receipt (the **old** branch being updated). Its contents go to `b_*[g]`.

`.we(t_we[g]), .w_idx(t_waddr[g]),`
`.w_valid(t_wvalid[g]), .w_tag(t_wtag[g]),`
`.w_ctr(t_wctr[g]), .w_u(t_wu[g]) );`
	The write port is driven by tage_update.

`tage_predict #( ... ) u_predict (`
`.t0_ctr_msb(t0_ctr_a[1]),`
`.t_valid(a_valid), .t_tag(a_tag),`
`.t_ctr(a_ctr), .t_u(a_u),`
`.hash_idx(hash_idx), .hash_tag(hash_tag),`
`.pred(pred_o), .meta(meta_flat) );`
	Place the predict logic. Inputs: T0's top bit and the port A contents of T1..T4, plus the hash results. Outputs: the prediction (directly to `pred_o`) and the 72-bit meta.

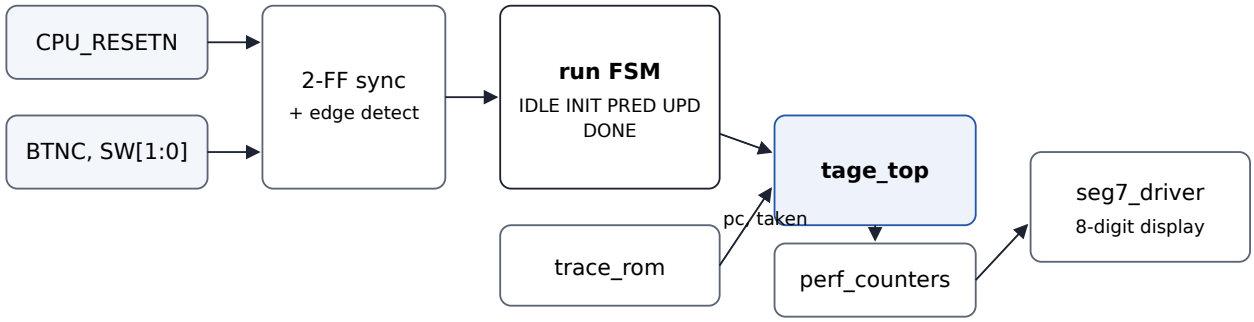
`assign meta_o = tage_meta_t'(meta_flat);`	Turn the 72 plain bits into the `tage_meta_t` struct (same order, so nothing changes in hardware).

`tage_update u_update (`
`.update_valid(update_valid_i),`
`.update_pc(update_pc_i),`
`.taken(update_taken_i), .meta(update_meta_i),`
`.rd_valid(b_valid), .rd_tag(b_tag),`
`.rd_ctr(b_ctr), .rd_u(b_u),`
`.t0_ctr(t0_ctr_b),`
`.t0_we(t0_we), ... , .t_wu(t_wu) );`
	Place the update logic. Inputs: the update pins, port B contents of T1..T4 and T0's counter for the old branch. Outputs: the write commands for T0..T4.

> **Why prediction and update can happen in the same cycle:** port A looks at the entry for the new branch, port B at the entry for the old branch. Each table has two read ports exactly for this.

5. The board demo, file by file

The FPGA has no CPU, so the board feeds the predictor test branches from a text file, counts the mistakes and shows them on the 7-segment display.



The board design (nexys7_top.sv).

5.1 branch_trace.mem: the test branches

rtl/board/branch_trace.mem	
`// ---- SW = 00 : loop  (28 lines)`	Lines starting with `//` are comments.
`@00`	"Put the next lines from memory address 0 on". The 4 patterns start at `@00`, `@20`, `@40`, `@60` (hex: 0, 32, 64, 96).
`1_00001040`	One branch: `1` = taken, `00001040` = its address (pc) in hex. The `_` is only for readability; the line is the 33-bit number {taken, pc}.
`0_00001040`	Same branch, not taken (the inner loop ends).

5.2 trace_rom.sv: plays the branches

Name	In / Out	Bits	What it carries
`clk, rst`	in	1, 1	clock; reset (1 = go back to the first line)
`mode`	in	2	which pattern (SW1 SW0)
`adv`	in	1	1 = go to the next line at the next clock edge
`valid`	out	1	1 once out of reset
`pc, taken`	out	32, 1	the current branch

rtl/board/trace_rom.sv	
`localparam logic [LEN_W:0] PAT_LEN [4] = '{28, 2, 20, 31};`	How many lines each pattern has (SW = 00, 01, 10, 11).
`logic [32:0] rom [4 << LEN_W];` `initial $readmemh(MEM_FILE, rom);`	A memory of 128 lines of 33 bits. `$readmemh` fills it from the text file when the FPGA is programmed.

`always_comb begin`
`  if (rst)  pos_next = '0;`
`  else if (!adv)  pos_next = pos;`
`  else if ({1'b0, pos} == PAT_LEN[mode] - 1'b1)`
`  pos_next = '0;`
`  else  pos_next = pos + 1'b1;`
`end`
	Which line comes next: after reset line 0; if not advancing, the same line; after the last line of the pattern, back to line 0; otherwise the next line.

`always_ff @(posedge clk) begin`
`  if (rst) begin pos <= '0; valid <= 1'b0; end`
`  else  begin pos <= pos_next; valid <= 1'b1; end`
`end`
	Store the line number at each clock edge.

`always_ff @(posedge clk) q <= rom[{mode, pos_next}];` `assign {taken, pc} = q;`	Read the line. The address is `{mode, pos}`: the 2 switch bits choose the block of 32 lines, `pos` the line in it. The 33 bits are split into `taken` (top bit) and `pc`.

5.3 nexys7_top.sv: the board controller

Name	In / Out	Bits	What it carries
`CLK100MHZ`	in	1	the board's 100 MHz clock
`CPU_RESETN`	in	1	red reset button (0 when pressed)
`BTNC`	in	1	centre button: start a run
`SW`	in	16	switches; SW[1:0] select the pattern
`LED`	out	16	LEDs (done, last prediction, progress bar, ...)
`CA..CG, DP, AN`	out	7, 1, 8	7-segment display: segments, dot, which digit is on

rtl/board/nexys7_top.sv: clock and inputs	

`MMCME2_BASE #(.CLKIN1_PERIOD(10.0),`
`  .CLKFBOUT_MULT_F(10.0), .CLKOUT0_DIVIDE_F(20.0))`
`  u_mmcm ( ... .LOCKED(mmcm_locked) ... );`
`BUFG u_bufg (.I(clk_mmcm), .O(clk));`
	A clock generator built into the FPGA: 100 MHz × 10 / 20 = **50 MHz**. The predictor needs about 16 ns per cycle, more than the 10 ns of 100 MHz. `BUFG` spreads the clock over the chip.

`lock_sync <= {lock_sync[0], 1'b1};`	Waits until the clock generator is stable (`LOCKED`); until then everything stays in reset.

`resetn_s <= {resetn_s[0], CPU_RESETN};`
`btnc_s  <= {btnc_s[0],  BTNC};`
`sw_s1  <= SW[1:0];`
`sw_s2  <= sw_s1;`
	Each button/switch goes through 2 flip-flops (a **synchroniser**), because they change at random times compared to the clock.

`assign rst = ~resetn_s[1] | ~lock_sync[1];`	Reset everything while the button is pressed or the clock is not ready.
`btnc_q <= btnc_s[1];` `assign start = btnc_s[1] & ~btnc_q;`	`start` is 1 for one clock when BTNC changes from 0 to 1 (now 1, one clock ago 0).

rtl/board/nexys7_top.sv: the run state machine	

`S_IDLE, S_DONE: if (start) begin`
`  soft_rst <= 1'b1;`
`  mode_q  <= sw_s2;`
`  state  <= S_INIT;`
`  end`
	Waiting. On BTNC: reset the predictor, trace and counters (`soft_rst`), remember the switches, go to S_INIT.

`S_INIT: if (!soft_rst && tage_ready && tg_valid)` `  state <= S_PRED;`	Wait one clock for the reset to finish, then start.

`S_PRED: if (done) state <= S_DONE;`
`  else begin`
`  p_tage  <= tage_pred;`
`  p_meta  <= tage_meta;`
`  p_pc  <= tg_pc;`
`  p_taken <= tg_taken;`
`  state  <= S_UPD;`
`  end`
	Cycle A (predict). The trace's pc goes into `tage_top`. Save the prediction, the receipt, the pc and the real outcome in registers (the pipeline register in Fig. A). After 40,000 branches go to S_DONE.

`S_UPD: state <= S_PRED;`	**Cycle B (update).** Lasts one clock, then the next branch.
`assign in_upd = (state == S_UPD);`	1 during cycle B. It drives the update, the counters and the trace advance.

`rtl/board/nexys7_top.sv: connections`	

`trace_rom #(.MEM_FILE(TRACE_FILE)) u_tg (`
`  .clk(clk), .rst(core_rst), .mode(mode_q),`
`  .adv(in_upd), .valid(tg_valid),`
`  .pc(tg_pc), .taken(tg_taken) );`
	The trace moves to the next line in every update cycle.

`(* dont_touch = "true" *)`
`tage_top u_tage (`
`  .pc_i(tg_pc), .pred_o(tage_pred),`
`.meta_o(tage_meta),`
`  .update_valid_i(in_upd), .update_pc_i(p_pc),`
`  .update_taken_i(p_taken), .update_meta_i(p_meta),`
`... );`
	The predictor. Predict side: the trace's pc. Update side: the saved values, in cycle B. `dont_touch` tells Vivado not to remove table entries that the few test PCs never use, so the size report is honest.

`perf_counters u_perf (`
`  .br_valid(in_upd),`
`  .tage_miss(p_tage != p_taken),`
`  .tage_cnt(tage_cnt), .br_cnt(br_cnt), .done(done)`
`);`
	In every update cycle: count one branch, and one mistake if the saved prediction differs from the real outcome.

`seg7_driver u_seg (.value({tage_cnt, br_cnt}), ...` `);`	Display: left 4 digits = mistakes, right 4 = branches (hex).

5.4 perf_counters.sv and seg7_driver.sv

`rtl/board/perf_counters.sv`	

`always_ff @(posedge clk) begin`
`  if (rst) begin`
`  tage_cnt <= '0; br_cnt <= '0; done <= 1'b0;`
	Reset: both counts 0, not done.

`  end else if (br_valid && !done) begin`
`  br_cnt  <= br_cnt + 16'd1;`
`  tage_cnt <= tage_cnt + {15'd0, tage_miss};`
	For every branch (until done): branches + 1, and mistakes + 1 if `tage_miss` is 1.

`  if (br_cnt == 16'(RUN_LEN - 1)) done <= 1'b1;`	After 40,000 branches: done, the counts freeze.

`rtl/board/seg7_driver.sv`	

`end else if (div == DW'(DIGIT_CYCLES - 1)) begin`
`  div  <= '0;`
`  digit <= digit + 3'd1;`
`end else div <= div + 1'b1;`
	The 8 digits share their segment wires, so only one digit is lit at a time. `div` counts 6,250 clocks (0.125 ms), then the next digit (0..7, then wraps). Too fast for the eye to see, so all 8 look lit.

`assign nib = value[{digit, 2'b00} +: 4];`	The 4 bits (one hex digit) for the current digit: digit 0 = bits 3..0, digit 1 = bits 7..4, ...

`case (nib)`
`  4'h0: seg = 7'h3F;`
`  4'h1: seg = 7'h06;  ...`
	Which segments to light for each hex digit 0..F (e.g. `3F` = segments a..f = "0").

`an_n  <= ~(8'b1 << digit);` `seg_n <= ~seg;`	Turn on only the current digit, and its segments. `~` because the board's display is on when the wire is 0.

6. Testing and results

6.1 How it is tested

- **tage_ref.sv** Member 4: a second, independent model of TAGE written in plain behavioural code: the "answer key".
- **tb_tage.sv**: runs `tage_top` and `tage_ref` side by side on many traces (loops, correlated branches, random branches, the 4 board patterns). On **every branch** the prediction and meta must match, and after every trace **every table entry** and the GHR must match.
- **tb/unit/**: one small test per module (table, hash, predict, update, counters).

```
Terminal
```

```
make sim      # all tests in Verilator   (make xsim: same in Vivado)
make build    # synthesis + bitstream -> bitstream/nexys7_top.bit
```

6.2 Results on the board

SW1 SW0	Pattern	Display	Meaning
00	loop	`000B 9C40`	11 mistakes in 40,000
01	alternating	`0002 9C40`	2 mistakes
10	correlated	`000C 9C40`	12 mistakes
11	mixed	`000E 9C40`	14 mistakes

All mistakes happen at the start, while the tables are still empty. After that every branch is predicted correctly. A simple counter without history could not learn the alternating or correlated patterns.

6.3 Hardware cost (Artix-7 xc7a100t)

Resource	Used	%
LUTs	10,186	16%
Flip-flops	15,587	12%
Clock	50 MHz, timing met (3.72 ns spare)	

The spec requires every table entry to be cleared in one reset, which FPGA RAM blocks cannot do, so the tables are built from flip-flops. That is why the flip-flop count is high and the clock is 50 MHz.